

Module 6 Week B — Core Skills Drill: Embedding Basics

Load pre-trained GloVe word vectors, compute cosine similarity between words, and find nearest neighbors in embedding space. This drill targets retrieval practice – write the code from the reading’s concepts, not by copying snippets.

Due: Tonight, after the concept session. Resubmissions are accepted through Saturday of the assignment week.

Setup

1. [Accept the assignment](#)
2. Clone your repo and open it in VS Code.
3. The repo contains `data/glove_50k_50d.txt` – a pre-extracted set of GloVe embeddings (50,000 words, 50 dimensions). Each line has a word followed by 50 space-separated float values.
4. Install dependencies:

```
pip install -r requirements.txt
```
5. Create a working branch:

```
git checkout -b drill-6b-embedding-basics
```

Tasks

Complete the functions in `drill.py`. The file contains function signatures with docstrings – you implement the logic. Run your code with `python drill.py` to verify outputs before pushing.

Task 1 – Load GloVe Vectors

Implement `load_glove(filepath)` :

- Open the file and read line by line
- Each line contains a word followed by 50 space-separated float values
- Parse each line into a word (string) and a vector (numpy array of shape `(50,)`)
- Return a dictionary mapping words to numpy arrays

What the autograder checks: The returned dictionary has approximately 50,000 entries. Each value is a numpy array with shape `(50,)` .

Task 2 – Compute Word Similarity

Implement `cosine_similarity(vec1, vec2)` :

- Compute the cosine similarity between two numpy vectors
- Handle the edge case where one or both vectors have zero norm (return 0.0)
- Return a float between -1 and 1

Test with known pairs from the reading:

- `cosine_similarity(glove["king"], glove["queen"])` should be greater than 0.5
- `cosine_similarity(glove["king"], glove["banana"])` should be lower than the king-queen similarity

What the autograder checks: Similarity for semantically related pairs is higher than for unrelated pairs. The function handles the zero-vector edge case without errors.

Task 3 – Find Nearest Neighbors

Implement `nearest_neighbors(word, embeddings, n=5)` :

- Look up the query word’s vector in the `embeddings` dictionary
- Compute cosine similarity between the query and every other word
- Return a list of `(word, similarity)` tuples for the top- `n` most similar words, sorted by similarity descending
- Exclude the query word itself from the results

What the autograder checks: Returns a list of the correct length. The results are sorted by similarity in descending order. For the query “king”, the word “queen” appears in the top 5.

Submission

1. Create a branch named `drill-6b-embedding-basics` .
2. Complete all three functions in `drill.py` .
3. Push your branch and open a PR to `main` .
4. Verify that the autograder passes (green check on the PR).
5. Paste your PR URL into TalentLMS -> Module 6 Week B -> Drill 6B to submit this assignment.